$addType\ (\mathbf{id}_3.entry, a_5)$;

$a_7 = a_5$;

$addType\ (\mathbf{id}_2.entry, a_7)$;

$a_9 = a_7$;

$addType\ (\mathbf{id}_1.entry, a_9)$;

这些语义规则的计算把类型 *integer* 存于符号表中每个标识符的条目中。 □

语义规则的这种计算方法称为**分析树方法**。若依赖图有环,则这种方法失败。这种方法的缺点是编译速度很慢,因为它是在编译过程中确定属性的计算次序。显然,要想提高编译速度,必须在编译前,即在构造编译器的时候就把属性计算次序确定下来,避免编译时构造依赖图和拓扑排序等工作。下面概述的两种方法分别是从手工构造编译器和自动生成编译器的角度考虑的改进。

在构造编译器时,用专门的工具或用手工对产生式的语义规则进行分析,对每个产生式,得到与它相关联的一组语义规则的计算次序。据此,把计算次序在编译前就确定下来。编译时,分析树上结点属性的计算就按事先确定的次序进行。这种方法称为**基于规则的方法**,它适用于手工构造编译器。这种方法的缺点是,一些属性依赖关系复杂的语法制导定义很难事先确定属性计算次序,因此这种方法对语法制导定义的种类有限制。

Yacc 等编译器生成工具都有确定的属性计算策略,如果编译器的设计者使用这样的工具来生成编译器,则必须按所选工具规定的计算策略去写语义规则。如果所写的语义规则不符合该计算策略对语义规则的约束,则属性计算不能正确完成。这种方法称为**忽略规则的方法**(oblivious method),即它不是根据语义规则的特点来选择计算策略,而是根据计算策略来限定编译器设计者所提供的语义规则的形式。这种方法大大限制了能够实现的语法制导定义的种类,但是能够得到高效的编译器。

基于规则的方法和忽略规则的方法都不必在编译时显式构造依赖图,和分析树方法相比,它们使编译器的时空效率大大改进。从 4.2 节开始介绍的各种具体翻译方法都属于这两种方法之一。其中 4.2 节介绍的综合属性自下而上计算方法最容易理解,也是编译器生成工具提供的基本方法。

4.2 S属性定义的自下而上计算

4.1 节初步介绍了如何用语法制导定义来规范翻译;从本节开始,将一边熟悉语法制导定义,一边研究其实现。对任意语法制导定义都适用的翻译器很难建立,但对于语法制导定义的一些常用形式,它们的翻译器很容易构造。本节以构造语法树为例,来熟悉 *S* 属性定义,并研究如何在自下而上的分析过程中完成属性计算。

4.2.1 语法树

语法树是分析树的浓缩表示。在语法树中，算符和关键字不是作为叶结点，而是作为分支结点。例如，对于产生式 $S \rightarrow$ **if** B **then** S_1 **else** S_2，可以把它的右部看成是有一个算符 **if-then-else** 和三个运算对象 B、S_1 和 S_2 的语言构造，它的语法树见图 4.4(a)。语法树中另一个简单的地方是单非产生式链可能消失，图 4.4(b)是表达式 8+5*2 的语法树。这两个例子的特点是，语法树上的内部结点都代表运算，其子结点都是它的运算对象。

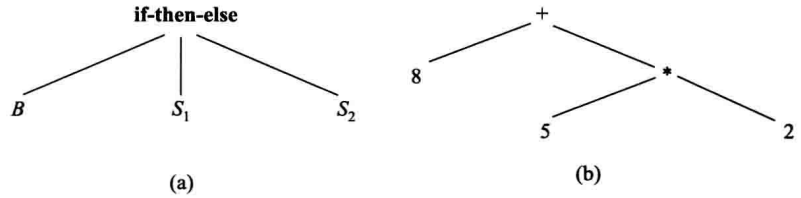


图 4.4 语法树的例子

语法树作为一种中间表示，允许把翻译从分析中分离出来，形成先分析后翻译的方式，即先分析生成语法树，然后再基于语法树进行翻译。即使是边分析边翻译，语法树作为一种概念上的中间表示，也是有用的。C 和 Java 的编译器通常显式构造语法树。

语法制导翻译可以基于分析树，也可以基于语法树，方法是一样的。如同在分析树中那样，在语法树中也可以把属性附加到结点。

4.2.2 构造语法树的语法制导定义

本节给出构造表达式的语法树的语法制导定义。首先介绍一下数据结构，语法树的结点可以用有若干域的记录来实现。对于算符结点，一个域存放算符，该域作为该结点的标记，其余两个域存放指向运算对象的指针。对于基本运算对象结点，一个域存放运算对象类型，另一个域存放其值。当真正用于翻译时，语法树的结点可能还有其他域来保存附加在该结点的其他属性值(或属性值的指针)。

下面解释语义规则中用到的函数，这些函数用来建立语法树的叶结点和分支结点，每个函数都返回新建结点的指针。

(1) *mkLeaf* (**id**, *entry*)。它建立标记为 **id** 的标识符结点；结点另有一个域，其值等于 *entry*，它是符号表中该标识符条目的指针。

(2) *mkLeaf* (**num**, *val*)。它建立标记为 **num** 的整数结点；结点另有一个域，其值等于 *val*，它是该整数的值。

(3) *mkNode* (*op*, *left*, *right*)。它用来建立标记为 *op* 的算符结点；结点另有两个域，其值分别

等于 *left* 和 *right*，它们是该结点左右子树的指针。

表 4.3 是为含+和 * 的表达式构造语法树的 S 属性定义。它给文法的产生式添加语义规则来安排对函数 *mkNode* 和 *mkLeaf* 的调用，以便构造语法树。E，T 和 F 的综合属性 *nptr* 用来记住函数调用返回的指针。

表 4.3 构造表达式语法树的语法制导定义

产生式	语义规则
$E \rightarrow E_1 + T$	$E.nptr = mkNode('+', E_1.nptr, T.nptr)$
$E \rightarrow T$	$E.nptr = T.nptr$
$T \rightarrow T_1 * F$	$T.nptr = mkNode('*', T_1.nptr, F.nptr)$
$T \rightarrow F$	$T.nptr = F.nptr$
$F \rightarrow (E)$	$F.nptr = E.nptr$
$F \rightarrow$ **id**	$F.nptr = mkLeaf$ (**id**, **id**. *entry*)
$F \rightarrow$ **num**	$F.nptr = mkLeaf$ (**num**, **num**. *val*)

例 4.6 图 4.5 给出了表达式 $a+5*b$ 的注释分析树和执行语义规则后所构造出的语法树，分析树由带点的线表示。标有非终结符 E，T 和 F 的分析树结点，其综合属性 *nptr* 指向由该非终结符推出的表达式对应的语法树的根结点。

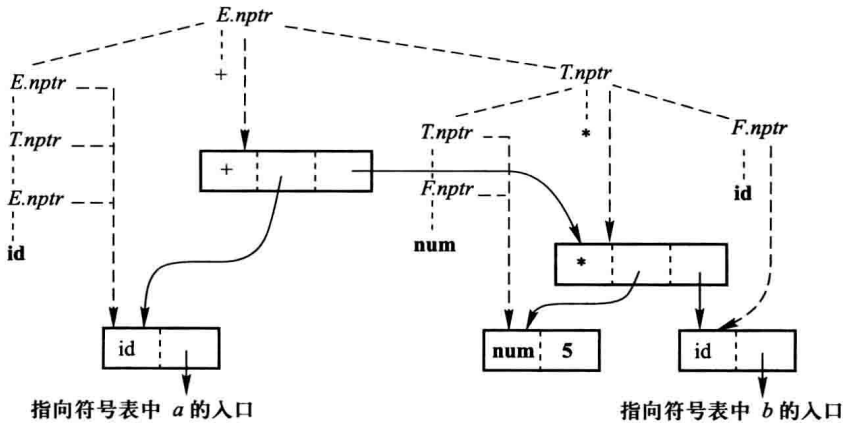


图 4.5 $a+5*b$ 的语法树的构造

产生式 $F \rightarrow$ **id** 和 $F \rightarrow$ **num** 的语义规则定义的属性 $F.nptr$ 也是指针，它们分别指向代表标识符和整数的叶结点。**id**. *entry* 和 **num**. *val* 是对应记号的属性值。

在图 4.5 中，当表达式 E 只有一项，也就是使用产生式 $E \rightarrow T$ 时，属性 $E.nptr$ 和 $T.nptr$ 的值一样。当使用产生式 $E \rightarrow E_1 + T$ 的语义规则 $E.nptr = mkNode('+', E_1.nptr, T.nptr)$ 时，先前的规则

已把 $E_1.nptr$ 和 $T.nptr$ 分别置为指向代表 a 的叶结点和代表 $5*b$ 的分支结点。

处于图 4.5 的下部，由记录形成的树是构成输出的真正语法树；而虚线表示的树是分析树，它可能仅有象征意义。 □

根据表 4.3 的语法制导定义，对于输入 $a+5*b$，实际执行的一系列函数调用如下：

(1) $p_1 = mkLeaf(\textbf{id}, entrya)$；

(2) $p_2 = mkLeaf(\textbf{num}, 5)$；

(3) $p_3 = mkLeaf(\textbf{id}, entryb)$；

(4) $p_4 = mkNode('*', p_2, p_3)$；

(5) $p_5 = mkNode('+', p_1, p_4)$；

可以看出，写语法制导定义比直接用编程语言写程序更困难。对程序而言，整个程序的执行流程是显式地用语句描述的。而语法制导定义中语义规则的执行受输入的语法制导，当输入串中的一个语法构造被识别时(可以看成发生一个事件)，其相应的语义规则被执行，因此本质上这是一种事件驱动的程序设计方法。

4.2.3 S 属性的自下而上计算

综合属性可以由自下而上的分析器在分析输入的同时完成计算。分析器可以把文法符号的综合属性放在栈中，每当归约时，根据出现在栈顶的产生式右部符号的属性来计算左部符号的综合属性。下面说明如何扩展分析器的栈，使之能够保存综合属性。在 4.4 节将看到这种实现也支持一些有继承属性的情况。

S 属性定义的翻译器可以借助 LR 分析器的生成器来实现，例如 3.7 节讨论的 Yacc。分析器的生成器根据 S 属性定义，可以构造出翻译器，它在分析输入的同时计算属性。

自下而上分析器用栈来保存已分析子树的信息。可以在分析栈中增加一个域来保存综合属性，图 4.6 给出了一个例子。拓展后分析栈的每个栈元素由状态域 *state* 和属性域 *val* 组成，或者说该栈由 *state* 和 *val* 两部分组成。为直观起见，图 4.6 的 *state* 部分用文法符号来代替状态，这个符号就是图 3.11 所描述的分析栈中被状态直接压住的那个符号。*val* 部分为文法符号存放综合属性。如果 *state* 部分某个位置的符号是 A，那么 *val* 部分相应位置保存分析树上对应这个 A 的结点的属性值。

栈顶由指针 *top* 指示，并假定综合属性刚好在每步归约前计算完成。若产生式 $A \rightarrow XYZ$ 的语义规则是 $A.a = f(X.x, Y.y, Z.z)$，那么在 XYZ 归约成 A 之前，属性 $Z.z$ 的值在 $stack[top].val$(指栈顶元素的 *val* 域)，$Y.y$ 的值在 $stack[top-1].val$(指次栈顶元素的 *val* 域，以下类推)，$X.x$ 的值在 $stack[top-2].val$。如果某个符号没有属性，那么其对应的 *val* 域没有定义。归约时，覆盖 A

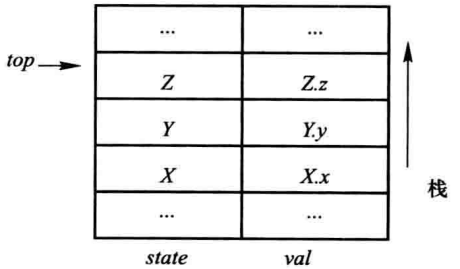


图 4.6 有综合属性域的分析栈

的状态放进 *stack*[*top*−2].*state*(即 *X* 原来的位置),综合属性的值 *A*.*a* 放进 *stack*[*top*−2].*val*,然后 *top* 的值减 2。

例 4.7 再次考虑表 4.1 计算器的语法制导定义。用 3.5 节的技术构造基础文法的 LR 分析器。为了计算属性,需要修改分析器,让它在归约前执行表 4.1 的语义规则,这些语义规则可以翻译成表 4.4 的栈操作代码段。这些代码段实际上就是通过用属性在栈中的位置代替表 4.1 语义规则中的属性而得到的。

表 4.4 用 LR 分析器实现计算器

产生式	表 4.1 的语义动作翻译成的代码段
$L\rightarrow E\mathbf{n}$	*print* (*stack*[*top*−1].*val*);
$E\rightarrow E_1+T$	*stack*[*top* −2].*val*=*stack*[*top* −2].*val* + *stack*[*top*].*val*;
$E\rightarrow T$	
$T\rightarrow T_1*F$	*stack*[*top* −2].*val*=*stack*[*top* −2].*val* × *stack*[*top*].*val*;
$T\rightarrow F$	
$F\rightarrow (E)$	*stack*[*top* −2].*val*=*stack*[*top* −1].*val*;
$F\rightarrow$**digit**	

表 4.4 中的代码段中,属性计算的结果都置入 *top*−2 的位置,因为刚好对应产生式的右部都是三个符号。这里的 *top* 指归约前的栈顶,归约后分析器会根据右部符号的多少调整它的值。注意,对应产生式 $E\rightarrow T$、$T\rightarrow F$ 和 $F\rightarrow$**digit** 没有代码段,因为相应语义规则的翻译结果是 *stack*[*top*].*val*=*stack*[*top*].*val*,显然可以忽略。

表 4.5 展示了面临输入 8+5 * 2**n**(注释分析树在图 4.1)时分析器的动作序列。每步动作后分析栈的 *state* 域和 *val* 域的内容都在表中给出,仍然用对应的文法符号代替状态。为直观起见,还用实际的输入数字代替记号 **digit**。

考虑看见第一个输入符号 8 时的动作序列。首先分析器把对应记号 **digit** 的状态和属性值移进栈(状态由 8 表示,属性值 8 在 *val* 域)。第二步,分析器用产生式 $F\rightarrow$ **digit** 归约并执行表 4.4 中对应的代码段。第三步,分析器按 $T\rightarrow F$ 归约,执行对应的代码段。第四步,分析器按 $E\rightarrow T$ 归约,执行对应的代码段。注意,没有代码段和这三个产生式相关联,所以 *val* 域一直不变,但每步归约后栈顶 *val* 域代表归约后产生式左部符号的属性。 □

表 4.5 翻译器面临 8+5 * 2n 时的动作

输入	*state*	*val*	所用产生式
8+5 * 2**n**	–	–	
+5 * 2**n**	8	8	

续表

输入	*state*	*val*	所用产生式
+5 * 2**n**	*F*	8	*F*→**digit**
+5 * 2**n**	*T*	8	*T*→*F*
+5 * 2**n**	*E*	8	*E*→*T*
5 * 2**n**	*E*+	8+	
* 2**n**	*E*+5	8+5	
* 2**n**	*E*+*F*	8+5	*F*→**digit**
* 2**n**	*E*+*T*	8+5	*T*→*F*
2**n**	*E*+*T* *	8+5 *	
n	*E*+ *T* * 2	8 + 5 * 2	
n	*E*+ *T* * *F*	8 + 5 * 2	*F*→**digit**
n	*E*+ *T*	8 + 10	*T*→*T* * *F*
n	*E*	18	*E*→*E* + *T*
	E **n**	18–	
	L	18	*L*→*E* **n**

在上面的实现轮廓中,代码段刚好在归约前执行。归约提供一种“挂钩”,任何代码段组成的动作都可以悬挂在上面,即允许用户把代码和产生式联系起来,当按此产生式归约时执行这些代码。

本节实际上揭示了3.7节介绍的生成器Yacc的语义动作的实现方法,对今后使用Yacc和类似的生成器会有很大帮助。

4.3 L属性定义的自上而下计算

从4.2节可知,将LR分析器进行拓展,很容易把*S*属性定义所要求的翻译嵌在分析过程中完成。那么,这种边分析边翻译的方式能否适用于有继承属性的情况呢?

若翻译在分析时发生,属性的计算次序一定受到分析方法所限定的分析树结点建立次序的限制。不管是自上而下分析还是自下而上分析,一个共同的特点是,分析树的结点总是自左向右地生成。如果属性信息是自左向右流动,那么就有可能在分析的同时完成属性计算。下面按这种想法来确定*L*属性定义,其中*L*表示左,因为属性信息是从左向右流动的。